Zenopher: Teacher Model

Status: Draft v1 **Scope:** How the teacher works: every check, how each one measures and scores a design, how findings are classified and fixed, how the checks are run, and how the AI critique fits in. **Relates to:** `zenopher-start-page.md` (editor levels and workflow), `zenopher-backend-free.md` (catalog, critique, consent, learning endpoints). Covers the checks, the teacher, the AI critique, style selection, and the engine that runs them.

---

1. What the teacher is

The teacher is a set of deterministic measurements and rules, plus an optional language-model critique.

- **Measurements** turn a document into numbers. They are pure functions of the document and never change it.
- **Rules** turn measurements into findings. A finding says what was measured, what was expected, how far off the design is, and how sure the teacher is.
- **Fixes** are optional commands attached to findings. The user decides whether to apply them.
- **Critique** is a single language-model review of the page image, given the findings as context. It produces text only.

Nothing in the teacher learns from the user's designs in v1. The thresholds and style profiles are fixed data, tuned by testing (Section 12). The only personalization is the familiarity profile, which controls how often hints appear, not what is measured.

---

2. Levels and what runs

Three editor levels, as decided in the start page spec:

Level	Checks that run	Findings shown	Fixes available
Creative	None	None	None
Corporate, no style	Base checks	Base findings	Base fixes
Corporate, with style	Base checks, style patterns, professional checks	All	All (safe fixes via Apply all)

Critique is available in Corporate at both levels (Section 9). It runs only when the user asks for it.

Check classes:

- **Base:** apply to any design, with or without a style. Measurements that are objective and tied to readability or correctness.
- **Pattern:** measured against a style profile's target ranges. Only run with a style.
- **Professional:** judgment-heavy checks, shown in Corporate with a style. Labeled as such in the UI.

---

3. The document inputs

Every check reads a snapshot of the document, not the live editor. The snapshot is produced by the engine.

rust

```rust
pub struct Snapshot {
    pub page: Page,                // width_px, height_px, background, short_sid
    pub layers: Vec<Layer>,        // see below, in stacking order
    pub style: Option<CompiledStyle>, // loaded and validated by the style stor
    pub mode: Level,               // creative, corporate_base, corporate_full
    pub intent: Intent,            // screen (default) or print, set in document
}

pub struct Layer {
    pub id: LayerId,
    pub kind: LayerKind,           // Text, Shape, Image, Group
    pub bounds: Rect,              // x, y, w, h in page px, after transforms
    pub visible: bool,
    pub opacity: f32,
    pub text: Option<TextInfo>,    // content, font_family, size_px, leading, tr
    pub fill: Option<Color>,
    pub image: Option<ImageInfo>, // source_w, source_h, displayed_w, displayed
}
```

Only visible layers are measured. Groups are measured through their children, and the group is not counted as an object.

3.1 Background sampling

Several checks need the colour behind a text layer. The engine computes it:

1. Render the page with the text layer hidden.
2. Sample pixels inside the text bounds, using a grid of at most 64 samples.
3. The background colour is the median of the samples in OKLab.
4. Measurement confidence is set from the spread of the samples:
    - spread below a low threshold: confidence 1.0 (flat background)

- spread above a high threshold, such as a photograph behind the text: confidence falls toward 0.4

Measurement confidence feeds the classification in Section 6.

3.2 Colour space

All colour reasoning uses OKLCH (lightness, chroma, hue), converted from sRGB through linear RGB and OKLab. Hue differences are measured on the circle, in degrees. Contrast uses WCAG relative luminance, which is the standard measure for text readability.

---

4. Measurements

Each measurement has an ID that style profiles refer to. The ID list is a fixed registry. A style profile that names an unknown measurement fails validation at build time.

ID	Unit	Definition
`contrast.text`	ratio (1 to 21)	WCAG contrast of each text layer's colour against its sampled background
`text.size_ratio`	ratio of font size to short side	Font size in px divided by the page's short side
`text.leading`	ratio	Line height divided by font size
`text.tracking`	em	Letter spacing in em
`text.count_sizes`	count	Distinct font sizes among text layers
`edge.distance`	px	Distance from each object's bounds to the nearest page edge
`edge.cropped`	boolean per object	Object bounds extend past the page
`gap.spacing_cv`	ratio	Coefficient of variation of gaps between neighbouring objects in a row or column
`overlap.area_ratio`	ratio	Intersection area of two objects divided by the smaller object's area
`align.deviation`	px	Difference between two objects' matching edges or centres
`tangency.gap`	px	Gap between two objects that nearly touch without overlapping
`colour.families`	count	Number of hue families among chromatic colours (Section 5.1)
`colour.dominant_share`	share of page area	Largest hue family's share of the page
`colour.neutral_share`	share of page area	Share of the page in colours with chroma below 0.04
`colour.accent_share`	share of page area	Share of the page in saturated colours outside the dominant family
`shape.geometric_share`	share of objects	Share of shape objects that are rectangles, ellipses, or regular polygons
`grid.edge_share`	share of edges	Share of object edges within tolerance of a column grid

ID	Unit	Definition
`balance.asymmetry`	ratio	Weighted centroid offset from the page centre (Section 5.9)
`hierarchy.ratio`	ratio	Largest text size divided by the second largest
`image.distortion`	ratio	Displayed aspect ratio divided by source aspect ratio, as a deviation
`image.aspect`	ratio	Displayed width divided by height, for proportion matching
`print.dpi`	dpi	Source resolution at print size (print intent only)
`print.bleed`	px	Presence and size of bleed outside trim (print intent only)

---

5. How each check works

Each check gives: its purpose, the measurement it uses, the algorithm, the output, its class, and its default confidence. Thresholds are starting values and are listed in the config (Section 11).

5.1 Colour family measurement (used by palette checks)

This is the basis for the palette checks, so it's defined first.

1. Downsample the page render to at most 128 × 128 px and drop fully transparent pixels.
2. Convert each pixel to OKLCH.
3. Discard pixels with chroma below 0.04. These are neutrals, counted separately.
4. For the remaining pixels, weight each by its area share.
5. Group hues into families: sort the chromatic hues, merge neighbours within 25°, and treat each group as one family. The family's hue is its area-weighted mean.
6. `colour.families` is the number of families holding at least 2% of the page area. Smaller families are noise.
7. `colour.dominant_share` is the page-area share of the largest family.
8. `colour.accent_share` is the share of pixels with chroma above 0.20 that are outside the dominant family.
9. `colour.neutral_share` is the share of pixels with chroma below 0.04.

The shares are measured against the page area, so a design with a large background colour counts that colour's family as dominant.

5.2 Contrast (base, must)

Purpose: text must be readable against what sits behind it.

Algorithm:

1. For each visible text layer with content, get its colour and sampled background (3.1).
2. Compute the WCAG ratio.
3. Required ratio: 3.0 for large text (font size at least 24 px, or at least 18.7 px and bold), otherwise 4.5.
4. A finding is raised when the ratio is below the required ratio.

Output: the layer, measured ratio, required ratio, and the background colour.

Class: base. **Default confidence:** must, when measurement confidence is at least 0.6. Otherwise optional.

Fix (safe): adjust the text colour's lightness in OKLCH, keeping hue and chroma where possible, in steps, until the ratio meets the requirement. If the required ratio can't be reached while keeping chroma, reduce chroma as needed. If it still can't be reached, the fix is marked "needs input" and offers black or white text instead.

5.3 Text size (base, must)

Purpose: text must be large enough to read at the design's intended size.

Algorithm:

1. For each visible text layer with content, compute its font size in px.
2. Compute `text.size_ratio = size_px / short_side`.
3. Minimum body ratio: 0.015 (for a 1080 px short side, about 16 px). Labels and captions are checked against the same minimum.
4. A finding is raised when the ratio is below the minimum.

Output: the layer, measured px, minimum px.

Class: base. **Default confidence:** must.

Fix (safe): set the font size to the minimum, rounded up to the nearest whole px. The layer's bounds are recalculated by the engine. If the layer then overflows the page, the crop check reports it.

5.4 Crop of text and logos (base, must)

Purpose: text and logos must not be cut off by the page edge.

Algorithm:

1. For each visible text layer and each layer tagged as a logo, test whether `bounds` extends past the page.

2. Any extension beyond the page is a finding. The measured value is the extension in px on each side.

Class: base. **Default confidence:** must.

Fix (safe): move the layer inside the page by the minimum translation that brings it fully inside. If the layer is larger than the page, the fix is "needs input" (scale it down, or accept the crop).

5.5 Crop of other objects (professional, optional)

Purpose: imagery cropped at the edge is often deliberate, so this is judgment, not an error.

Algorithm: same as 5.4 for shapes and images, reporting how much of the object is cut off as a share of its area.

Class: professional. **Default confidence:** optional. Never auto-fixed.

5.6 Alignment (base, must for near misses)

Purpose: objects that are almost aligned look accidental. Objects that are clearly aligned are fine, and objects that are far apart are unrelated.

Algorithm:

1. Consider pairs of visible objects whose bounds are within a neighbourhood of each other, defined as a gap of at most 5% of the short side in one axis and overlapping projections in the other.
2. For each pair, compare these edge pairs: left with left, right with right, centre-x with centre-x, top with top, bottom with bottom, centre-y with centre-y.
3. For each pair, take the smallest difference `d`.
4. A near miss is raised when `0 < d ⩽ tolerance`, where tolerance is 0.2% of the short side (2 px at 1080 px). Exact matches (`d = 0`) are aligned and raise nothing. Differences above tolerance are intentional or unrelated and raise nothing.

Output: the pair, the edge that matches, and `d` in px.

Class: base. **Default confidence:** must, since the threshold is objective. The explanation says the objects are almost aligned, which is a near miss, and not that the design is wrong.

Fix (safe): move the object with the smaller weight (the one not anchored by the page or a larger object) so its matching edge equals the other object's edge.

5.7 Spacing and margin (base for consistency, pattern for margin)

Spacing consistency:

1. Group objects into rows and columns: objects whose vertical (or horizontal) projections overlap form a row (or column).

2. In each group of three or more objects, sort them and compute the gaps between neighbours.
3. `gap.spacing_cv` is the standard deviation of the gaps divided by their mean.
4. A finding is raised when the CV is above 0.15 for a group, measured in px.

Class: base. **Default confidence:** optional for groups of three objects, must for groups of five or more, since larger groups make the inconsistency more visible and less likely to be intentional.

Fix: none. The teacher suggests the mean gap as a target, and the user adjusts. Equalizing gaps can break other intentional layout, so it is never automatic.

Margin (pattern): `edge.distance` for the outermost objects is compared with the profile's margin target (for example 5% of the short side for Bauhaus). Scored with the pattern scoring in 5.12.

5.8 Overlap (base, split by case)

Purpose: text that collides with text is almost always a mistake. Shapes overlapping shapes is often intentional.

Algorithm: for each pair of visible objects, compute `overlap.area_ratio`.

- **Text over text** with `overlap.area_ratio` above 0.10: finding.
- **Text over a shape or image** is treated as intentional, and checked by contrast (5.2) instead.
- **Shape over shape** with ratio above 0.25: optional finding, labeled "overlapping shapes". Not auto-fixed.

Class: base for text over text. **Default confidence:** must for text over text, optional for shapes.

Fix (safe, text over text only): move the later text layer down or sideways by the minimum amount that clears the overlap, if that stays on the page. Otherwise "needs input".

5.9 Balance (professional, pattern, optional)

Purpose: a design can be symmetric, or it can be asymmetric with the weight shifted deliberately. The style decides which is wanted.

Algorithm:

1. Weight each visible object: `weight = area × visual_strength`, where visual strength is the contrast of the object against the page background (3.1), scaled to 0 to 1, with images counted at their mean contrast.
2. Compute the weighted centroid `c` of all objects.
3. `balance.asymmetry = |c - page_centre| / (page_half_diagonal)`.
4. Scored against the profile's target range (for Bauhaus, 0.1 to 0.5).

Output: the centroid, the asymmetry, and the target range.

Class: professional, pattern. **Default confidence:** optional. Never must.

Fix: none. The teacher shows the centroid position on the canvas, not a move.

5.10 Tangency (professional, optional)

Purpose: two objects that nearly touch, without overlapping or aligning, look like a mistake.

Algorithm: for each pair of visible objects that do not overlap, compute the gap between their bounds. A tangency is raised when the gap is greater than 0 and at most 6 px, and the pair is not aligned (5.6 raises nothing for them).

Class: professional. **Default confidence:** optional.

Fix: none automatically. Suggest either aligning or separating by at least 12 px.

5.11 Visual hierarchy (pattern, optional)

Purpose: a clear primary text and a clear secondary text, not many competing sizes.

Algorithm:

- `hierarchy.ratio` = largest font size ÷ second largest distinct font size.
- `text.count_sizes` = number of distinct font sizes.
- Scored against the profile's targets, for example a ratio of at least 1.5 and at most four sizes.

Class: pattern. **Default confidence:** optional.

Fix: none. The teacher names the sizes it found and the target.

5.12 Pattern scoring (all pattern checks)

Each pattern has a measurement `m`, a target range `[lo, hi]`, and a tolerance `t`, from the style profile.

```
distance = 0            if lo ≤ m ≤ hi
         = lo − m       if m < lo
         = m − hi       if m > hi

score    = max(0, 1 − distance / t)
```

A score of 1.0 is inside the range. A score of 0 is at least one tolerance outside it. Pattern findings always carry a score and never a pass/fail verdict.

Overall fit (optional display) is the weighted mean of the pattern scores, with weights from the profile. The UI shows it only if the product decision on overall score is made.

5.13 Colour patterns (pattern, optional or must)

These use the measurements from 5.1, scored by 5.12:

- `colour.families` against the profile's target range (Bauhaus: 1 to 3)
- `colour.dominant_share` (Bauhaus: 0.4 to 0.8)
- `colour.neutral_share` (Bauhaus: 0.15 to 0.5)
- `colour.accent_share` (Bauhaus: 0 to 0.2)

Class: pattern. **Default confidence:** optional. Colour patterns describe a palette, and a palette is a judgment, so they are never must.

Fix: none. The teacher shows the measured families and shares, and what the style expects.

5.14 Style shape and grid patterns

- `shape.geometric_share`: share of shapes that are regular geometry. Scored against the profile's target. Optional.
- `grid.edge_share`: share of object edges within tolerance of the profile's column grid. The grid is the page width minus margins, divided into the profile's column count, with gutters. Optional.

Class: pattern. **Fix:** none.

5.15 Proportion and image distortion (base for distortion, professional for ratios)

Distortion (base, must): `image.distortion` = displayed aspect ÷ source aspect. A deviation above 2% is a finding, because a stretched photograph is almost always an accident.

Fix (safe): restore the source aspect by adjusting the displayed height to match the width, keeping the object's centre.

Ratio matching (professional, optional): `image.aspect` compared with the profile's preferred ratios. Scored by 5.12.

5.16 Leading and tracking (base for leading, professional for tracking)

Leading (base, must): `text.leading` below 1.0 causes lines to collide, and above 2.0 breaks the paragraph apart. Both are findings.

Fix (safe): set leading to 1.2 for body text, the middle of the acceptable range.

Tracking (professional, optional): tight tracking on lowercase body text, or very wide tracking on large text, is raised as optional. Not auto-fixed.

5.17 Expert setup for print (print intent only, base, must)

Runs only when the document's intent is print. Screen-only designs skip it.

- `print.dpi` below 300 for raster images at print size: must.

- `print.bleed` absent or below 3 mm equivalent: must.
- Colour mode is RGB while the intent is print: optional, since the conversion depends on the print workflow.

Fix: none for DPI or bleed. Explain what is needed and where to fix it.

---

6. Classification: must, optional, and measurement confidence

Two separate confidence measures. They are often confused, so they are defined apart.

Severity (what the UI shows):

- **Must change:** an objective violation, such as contrast below the required ratio, text below the minimum size, or text cropped by the page.
- **Optional:** a judgment or a pattern score. Shown in the list, never counted toward the badge.

Measurement confidence (how sure the teacher is that the measurement is right):

- Between 0 and 1. Set by the measurement itself: background sampling spread (3.1), the number of objects in a group, whether an image's source size is known.
- A finding with measurement confidence below 0.6 is downgraded from must to optional, and its explanation says the teacher is unsure.

Rules, in order:

1. Pattern findings are never must.
2. Professional findings are never must.
3. A base finding is must when its measurement confidence is at least 0.6 and the check's default is must.
4. Otherwise a base finding is optional.

This keeps the "must change" list short and defensible, and gives the "optional" list room for judgment.

---

7. Findings

Every check returns findings in one shape:

```rust
```

```rust
pub struct Finding {
    pub rule: RuleId,              // e.g. "contrast", "colour.dominant_share"
    pub class: Class,              // base, pattern, professional
    pub severity: Severity,        // must, optional
    pub layers: Vec<LayerId>,      // stable, for "Show" and for dedupe
    pub measured: Measured,        // value and unit
    pub expected: Expected,        // a threshold, a range, or a minimum
    pub score: Option<f32>,        // pattern findings only, 0 to 1
    pub measurement_confidence: f32,
    pub message: MessageKey,       // plain-language template, localized later
    pub explanation: MessageKey,   // the ⓘ text
    pub fix: Option<Fix>,
}

pub struct Fix {
    pub safety: FixSafety,         // safe, needs_input
    pub commands: Vec<Command>,    // engine commands, applied as one batch
    pub conflict_key: String,      // e.g. "text:12:colour", for conflict detect
}
```

Finding keys are `(rule, layers)`. A finding keeps its key across recomputation, so dismissals and the UI's position in the list stay stable.

7.1 Message rules

- Each message states the measured value and the expected value. "Contrast is 2.1:1. Needs 4.5:1."
- Each message is one sentence of cause and one of effect, at most.
- No message says "bad", "wrong", or "ugly." The teacher describes the measurement and the consequence.
- Explanations (the ⓘ text) say why the rule exists: readability, print reproduction, consistency.

---

8. Orchestration

8.1 When checks run

Trigger	Corporate, no style	Corporate, with style
Edit committed (after 400 ms idle)	Base checks on affected objects	Base, pattern, and professional checks on affected objects
Style attached or removed	Nothing new (base unaffected)	Pattern and professional checks added or removed
Mode switched into Corporate	Full base run	Full run
Mode switched to Creative	Findings cleared	Findings cleared
Document opened	No run until the user edits or switches into Corporate	Same
Critique requested	Runs with base findings	Runs with all findings

Creative runs nothing, not even on open, so the teacher never interrupts pure design.

8.2 Incremental recomputation

- A change to layer `L` invalidates: every finding whose `layers` contains `L`, every pair involving `L` for pairwise checks (alignment, overlap, tangency, spacing groups), and the colour measurements if `L` is visible and has a fill or image.
- Global measurements (colour families, balance centroid) are recomputed once per run, not per layer.
- Pairwise checks are limited to objects within a neighbourhood, so the cost grows with nearby pairs, not with all pairs.

Target: a full run on a page with 50 objects completes in under 20 ms in the browser's WASM engine. The colour measurement dominates, so it runs on the downsampled render.

8.3 Stable ordering and dismissals

- Findings are sorted by class (base, then pattern, then professional), then severity (must, then optional), then rule, then layer order.
- A dismissed finding stays hidden until a layer in its `layers` changes. The dismissal stores the key and the layer versions.
- A finding that disappears because the user fixed it is removed. One that reappears after a change is shown again.

8.4 Apply, and Apply all

Single fix: runs the finding's commands as one undoable step, then re-runs the checks for the affected layers.

Apply all:

1. Collect the fixes of all findings with severity must and safety safe. Optional findings and "needs input" fixes are excluded.
2. Group by `conflict_key`. Where two fixes touch the same property, keep the one from the higher-severity finding, and list the other as "conflicts with another fix", not applied.
3. Show the summary: "Apply N fixes: change text colour on 2 layers, move 1 layer 6 px, set 1 font size."
4. On confirm, run all commands as one batch, so one undo restores everything.
5. Re-run all checks. If new must findings with safe fixes appear, repeat, for at most three passes. Stop when no new safe must findings appear. Show any remaining findings as they are.

The three-pass limit prevents two fixes from undoing each other forever.

8.5 Style changes

Attaching or removing a style changes which checks run, not the document. Existing base findings keep their keys and stay visible. Pattern and professional findings appear or disappear accordingly.

---

9. The AI critique

9.1 Purpose

The rules measure what can be measured. The critique gives a second reading of the whole page, as a designer would, covering readability in context, emphasis, balance, and spacing as a viewer sees them. It catches things the rules miss and frames the rules' numbers in plain language.

9.2 What it is not

- It is not a source of measurements. It doesn't produce numbers that the UI treats as findings.
- It cannot create a finding with a fix.
- It cannot change the document.
- It is not run automatically. It runs when the user asks.

9.3 Inputs

1. **Page image:** a PNG of the page at most 1024 px on the longest side, rendered without selection handles or guides.

2. **Page facts:** size, mode, and style name (if set). The style name is sent only when a style is set.
3. **Findings summary:** a compact list of current findings, each with its rule name, class, severity, measured value, and expected value. No layer IDs and no document content beyond what the image shows. Text content is included only as the words already visible in the image.

9.4 Prompt structure

The prompt has four fixed parts:

1. **Role:** a design reviewer for a teaching tool, speaking to a beginner.
2. **Task:** review the page image against the findings and the style, in four areas: readability, emphasis, balance, spacing.
3. **Rules for the reply:**
   - Refer to findings by their rule name. Don't invent new measurements.
   - If a finding exists, the reply may agree or add context, but must not contradict a measured value.
   - Keep every point to one or two sentences.
   - Don't suggest specific pixel values or command names.
   - Reply only in the output format.
4. **Output format:** a JSON object, validated by the server.

The findings summary and the page facts are placed in the prompt as data, clearly delimited, so text inside the design can't be mistaken for instructions.

9.5 Output format

```json
{
  "summary": "one or two sentences",
  "points": [
    {
      "area": "readability | emphasis | balance | spacing",
      "agrees_with": ["contrast"],
      "comment": "one or two sentences"
    }
  ]
}
```

The server rejects anything that isn't this shape, caps the number of points at 6, caps each comment at 240 characters, and removes markup. A rejected reply is retried once with a stricter instruction, then reported as unavailable. A retry costs credits, so it's counted in the budget.

9.6 How the critique is shown

- Points appear under the findings, in a separate section labeled "Review".
- Each point shows its area. If it references a finding, it shows a "Show" link to that finding.
- Points never have a Fix button.
- The section is marked as AI text, and the user can hide it.

9.7 Checks on the critique itself

- **Consistency:** when the critique says something is readable and a contrast finding exists, the disagreement is shown as such, not hidden.
- **Length and shape:** enforced by the server (9.5).
- **Injection:** text inside the design can attempt to change the reply. The output format is fixed, and any reply outside it is discarded.

9.8 When it runs, and cost

- One request per user action, "Review my design". This is the "fires all at once" behaviour from the notes: one action runs every check and the critique together.
- Budget and per-user limits are in the backend spec.
- Results for the same image are cached for 24 hours, so reopening the review costs nothing.

9.9 Fallback without image support

If the chosen model cannot take images, the critique input becomes the page facts and the findings summary only. The reply is then limited to the findings, and the "Review" section says it is based on measurements, not on what the page looks like. Confirm the model's capabilities before choosing this path (backend spec, 9.1).

---

10. Testing and calibration

The rules are only useful if their findings match a designer's judgment. Testing is part of the design.

10.1 Fixtures

For each rule, build a set of small documents:

- **Positive:** designs that should raise the finding, with the expected measurement.
- **Negative:** designs that should not, including intentional cases (text over an image is fine if contrast is met, aligned objects are fine, exact matches are fine).
- **Boundary:** values just inside and just outside each threshold.

Fixtures are stored in the repository as `.zph` files with expected results in JSON. Rust tests run them on every change.

10.2 Labelled designs

Collect 50 to 100 designs and have a designer label each finding as correct, wrong, or debatable. This is the calibration set. It drives threshold tuning and the confidence rules.

Acceptance:

- Must findings: precision at least 0.9. A must finding that a designer calls wrong is a bug in the threshold or the measurement.
- Optional findings: precision at least 0.6. Optional findings are allowed to be debatable, but not routinely wrong.
- Recall for base checks: at least 0.9 on the calibration set.

10.3 Measurement tests

- Colour measurement: known palettes with known family counts and shares.
- Background sampling: flat background, gradient, photograph, and text over a shape.
- Contrast: the WCAG reference values, including the boundary for large text.
- Alignment: pairs at 0, 1, 2, and 3 px apart, checking the tolerance boundary.

10.4 Regression

Every change to a threshold, a measurement, or a style profile reruns the fixture suite and the calibration set. A change that reduces precision on must findings is rejected, even if it improves other numbers.

10.5 Style profile validation

- Every measurement referred to exists in the registry.
- Every target range has `lo ⩽ hi`, and a tolerance above zero.
- Every profile is tested against a small set of reference designs in that style, to check the scores land where a designer would expect.

---

11. Configuration

All thresholds are data, not code, so they can be tuned without a release of the engine.

```json
```

```
{
  "contrast": { "large_text_px": 24, "large_bold_px": 18.7, "required_large": 3
  "text_size": { "min_ratio_of_short_side": 0.015 },
  "leading": { "min": 1.0, "max": 2.0, "fix_to": 1.2 },
  "alignment": { "neighbourhood_ratio": 0.05, "tolerance_ratio": 0.002 },
  "spacing": { "cv_threshold": 0.15, "must_min_objects": 5 },
  "overlap": { "text_text_ratio": 0.10, "shape_shape_ratio": 0.25 },
  "tangency": { "max_gap_px": 6, "suggest_gap_px": 12 },
  "distortion": { "max_deviation": 0.02 },
  "colour": { "chroma_neutral": 0.04, "chroma_accent": 0.20, "hue_merge_deg": 2
  "confidence": { "must_min_measurement": 0.6 },
  "orchestration": { "idle_ms": 400, "apply_all_max_passes": 3 },
  "critique": { "max_image_px": 1024, "max_points": 6, "max_comment_chars": 240
}
```


A style profile can override any base threshold it needs, within a stated range. Overrides outside the range are rejected at validation.

---

12. Worked example

A 1080 × 1080 Corporate design with a style. Two text layers, a headline and a caption, sit over a shape, and a photo sits at the bottom.

1. **Headline contrast.** Text colour light grey, background sampled as mid blue. Ratio 2.9:1. Size 72 px, so large text, required 3.0. Measurement confidence 1.0 (flat background). **Must.** Fix: darken the text colour in OKLCH, keeping hue, until the ratio reaches 3.0. Safe.
2. **Caption size.** 14 px, minimum is 16.2 px (0.015 × 1080). **Must.** Fix: set to 17 px, rounded up. Safe.
3. **Caption and photo alignment.** Caption left edge at 120 px, photo left edge at 122 px. Difference 2 px, within tolerance. **Must.** Fix: move the caption (the object not anchored by the page) to 122 px. Safe.
4. **Dominant colour share.** Blue covers 0.55 of the page. Bauhaus target 0.4 to 0.8. Score 1.0. Nothing shown.
5. **Accent share.** Saturated yellow covers 0.27. Target 0 to 0.2, tolerance 0.1. Distance 0.07, score 0.3. **Optional**, pattern. Shown with the measured share and the target. No fix.
6. **Balance.** Asymmetry 0.08. Target 0.1 to 0.5, tolerance 0.15. Distance 0.02, score 0.87. **Optional**, professional. Shown with the centroid.
7. **Photo distortion.** Displayed aspect 1.0, source aspect 1.33. Deviation 25%. **Must.** Fix: restore the source aspect by adjusting the height, keeping the centre. Safe.

Apply all applies fixes 1, 2, 3, and 7 as one batch, after the summary. Finding 5 and 6 remain in the list as optional. The re-run after the batch shows no new must findings, so the process stops after one pass.

---

13. Style selection

13.1 Where a style is chosen

Place	Behaviour
Set up design, Corporate step	Style picker shown. Skip is a normal, equal-weight option.
Set up design, Creative	No style step.
Editor, style chip (top bar)	Opens the picker. Available in both modes.
Learn > Style library	Browse only. Choosing a card here opens the picker with a "Use in new design" action.

Choosing a style in Creative attaches it, but has no effect until the user switches to Corporate. Attaching a style is a document change and is undoable. Switching mode is not (start page spec, Section 5).

13.2 Picker contents

Each card shows:

- the example image (the first example in the profile)
- the style name
- the one-line summary from the profile
- the family and use tags

The picker supports search by name and filtering by family and use. It reads the catalog index, which is cached on the device and refreshed at most once a day.

Fit preview (optional, decision 19.4): when a page already has content, each card may show "Fit" for that page. This runs the pattern checks against each style, which costs time. If built, it runs on demand, one style at a time, and is labelled as a measurement, not a verdict.

13.3 Attach, detach, and version commands

Style changes are engine commands, so they go through the same history as other edits:

Command	Effect	Undoable
`style.attach { style_id, version }`	Sets the document's style	Yes
`style.detach`	Clears the document's style	Yes
`style.set_version { version }`	Moves the design to a newer version of the same style	Yes

The document stores the style as metadata, not content:

```json
"style": { "id": "bauhaus", "version": 3 }
```

or `null` when no style is attached. The `.zph` file carries this field, so reopening restores the style.

Changing the style re-runs the checks. Base findings keep their keys and stay visible. Pattern and professional findings appear or disappear accordingly (teacher model, Section 8.5).

13.4 Versions and pinning

- A design is pinned to the style version it was checked against.
- A newer published version does not change existing designs.
- The panel shows "Style update available" when a newer version exists. The user chooses to apply it with `style.set_version`, which is undoable.
- Unpublishing a version does not remove it. Published versions are archived, marked `deprecated`, and kept while any design references them. This replaces the "Style no longer available" state in the earlier specs for normal cases.

13.5 Missing styles

A style can be unavailable for three reasons. Each has its own message and never blocks editing.

Reason	Message in the panel	Style checks
Not downloaded yet and offline	"Style not downloaded. Connect to load it."	Off
Version not found in the catalog (should not happen; logged as an error)	"This style version can't be loaded. Choose another style."	Off
Profile failed validation after an app update	"This style needs an app update."	Off

Base checks are unaffected in all three cases.

---

14. Style profile

14.1 Format

```json
```

```
{
  "id": "bauhaus",
  "version": 3,
  "name": "Bauhaus",
  "summary": "Geometric forms, a strict grid, a few hues, and large colour bloc
  "families": ["geometric", "print"],
  "uses": ["poster", "social"],
  "principles": [
    "Geometric forms are the main visual vocabulary.",
    "Layout follows a visible grid, often asymmetric."
  ],
  "examples": [
    {
      "image": "bauhaus/poster-1.webp",
      "source_url": "https://example.org/source",
      "license": "CC-BY-4.0",
      "attribution": "Author, Title, Year"
    }
  ],
  "patterns": {
    "colour.families":       { "target": [1, 3],   "tolerance": 1,   "weight":
    "colour.dominant_share": { "target": [0.4, 0.8], "tolerance": 0.2, "weight"
    "colour.neutral_share":  { "target": [0.15, 0.5], "tolerance": 0.15, "weigh
    "colour.accent_share":   { "target": [0, 0.2], "tolerance": 0.1, "weight":
    "shape.geometric_share": { "target": [0.7, 1.0], "tolerance": 0.2, "weight"
    "grid.edge_share":       { "target": [0.7, 1.0], "tolerance": 0.2, "weight"
  },
  "professional": {
    "balance.asymmetry": { "target": [0.1, 0.5], "tolerance": 0.15, "weight": 0
  },
  "overrides": {
    "spacing.cv_threshold": 0.12
  },
  "status": "published",
  "content_hash": "sha256:..."
}
```

Field rules:

- `principles` are descriptive text for the user. The engine never evaluates them. Measurable patterns are the only thing that is scored, which keeps styles from becoming strict rulebooks.
- `patterns` use measurement IDs from the registry (teacher model, Section 4). Each has a target range, a tolerance, and a weight.

- `professional` holds judgment-based patterns. They are shown only in Corporate with a style, and are never must.
- `overrides` change base thresholds for this style. Each override key has an allowed range defined in the engine. An override outside that range fails validation.
- `examples` carry their licence and attribution. Examples without a licence that allows use are rejected at publishing.
- `content_hash` identifies the exact profile. Caches use it, so a changed profile never reuses old data.

14.2 Validation

The engine's style loader runs these checks before a profile can be used:

1. Schema: all required fields present, correct types, no unknown fields.
2. Measurement IDs: every key under `patterns`, `professional`, and `overrides` exists in the registry.
3. Ranges: every target has `lo ⩽ hi`, every tolerance is above zero, every weight is between 0 and 1.
4. Overrides: each value is inside its allowed range.
5. Hash: the content matches `content_hash`.
6. Examples: each has a licence and attribution.

A profile that fails any check is not used. The panel shows the "needs an app update" or "can't be loaded" message (13.5), and the failure is logged without content.

14.3 Compiled form

After validation, the loader compiles the profile into a read-only form that the rules use directly:

```rust
pub struct CompiledStyle {
    pub id: StyleId,
    pub version: u32,
    pub hash: ContentHash,
    pub patterns: Vec<CompiledPattern>,       // lo, hi, tolerance, weight, clas
    pub professional: Vec<CompiledPattern>,   // class = professional
    pub overrides: ThresholdOverrides,        // applied on top of base config
}
```

Compiled styles are cached in memory by hash. A style is compiled once per session.

---

15. The teacher engine

15.1 Components

```
Shells (React)
   ↕  typed bridge: commands, queries, subscriptions
Engine (Rust → WASM)
   ├── Document core        layers, history, commands (existing engine)
   ├── Render cache         page render, per-layer render versions
   ├── Teacher
   │     ├── Scheduler      idle timer, gesture gate, time-slicing
   │     ├── Snapshot       builds the input for a run, from changed layers
   │     ├── Measurements   registry, incremental caches
   │     ├── Rule runner    runs each rule in isolation
   │     ├── Finding store  diffs runs, holds dismissals and order
   │     ├── Fix planner    single fix, Apply all, pass limit
   │     └── Hint policy    display-only adjustments from familiarity
   └── Style store          loads, validates, compiles, caches profiles

Browser services (TypeScript)
   ├── Catalog sync         index and profile cache in IndexedDB
   ├── Critique client      builds the request, shows results
   ├── Event recorder       consent-gated, batches and uploads outcomes
   └── Persistence          autosave to IndexedDB, .zph import and export
```

Everything that decides what is wrong runs in the engine. The browser services only move data to and from the backend, or display results.

15.2 Public teacher API

The shells use only these calls. Nothing else reaches into the teacher.

Call	Purpose
`teacher.setLevel(level)`	Set creative, corporate_base, or corporate_full. Creative clears findings.
`teacher.setStyle(styleRef or null)`	Used when the style is attached or detached (also a command).
`teacher.subscribe(callback)`	Receives finding diffs: added, changed, resolved.
`teacher.run(scope)`	Forces a run for the whole page or selected layers. Used on mode switch and style change.
`teacher.apply(findingKey)`	Applies one safe fix, as one undoable step.
`teacher.applyAll()`	Returns the summary. The shell confirms, then calls `teacher.commitApplyAll(plan)`.
`teacher.dismiss(findingKey)`	Hides a finding until its layers change.
`teacher.explain(findingKey)`	Returns the ⓘ text.
`teacher.show(findingKey)`	Selects the affected layers on the canvas.

`applyAll` is split into two steps so the user sees the summary before anything changes.

15.3 How a run works

1. **Command applied.** The document core records the command in history and emits a change event with the affected layer IDs and their new versions.
2. **Gate check.** If a gesture is active (drag, transform, text typing), the scheduler records the pending change and waits. Nothing runs.
3. **Idle window.** When the gesture ends, the scheduler waits 400 ms with no further changes. Changes during that window extend it.
4. **Snapshot delta.** The snapshot builder collects the changed layers and any layers paired with them (for alignment, overlap, tangency).
5. **Measurements.** Cached measurements for unchanged layers are reused. Changed layers are re-measured. Page-wide measurements (colour families, balance) are recomputed once if anything changed.
6. **Rules.** Each rule runs on the snapshot. A rule that throws is isolated: its findings are marked unavailable, and other rules continue.
7. **Diff.** The finding store compares the new findings with the stored ones by key and produces added, changed, and resolved sets.
8. **Hint policy.** Display-only adjustments are applied to the diff (15.5, Section 16).

9. **Emit.** The diff goes to subscribers. The UI updates the badge and the affected rows, and nothing else.

No run writes to the document, the history, or the event log. Only applied fixes and dismissals are recorded.

15.4 Caches

Cache	Key	Invalidated by
Layer measurements	layer ID and layer version	that layer changing
Page colour histogram	page render hash	any visible layer changing
Text background samples	text layer version and the versions of layers beneath it	any of those layers changing
Compiled style	content hash	never, within a session
Finding store	finding key	the finding's layers changing

Caches are inside the engine. They never leave it and are discarded on document close.

15.5 Time budget and threading

- A full run on a page with about 50 objects should finish within 20 ms of measurement time, not counting rendering (teacher model, Section 8.2).
- The colour histogram is the most expensive step. It runs on the downsampled render (at most 128 × 128 px).
- If a run exceeds a 12 ms slice, the rest is continued on the next idle frame. The panel may update in two steps. Input is never blocked.
- The engine runs on the main thread in the MVP. Moving measurements to a Web Worker is a later option. It needs cross-origin isolation (the backend security headers already note this), so it is not assumed here.

15.6 Determinism

The same snapshot must produce the same findings in the same order. The engine contains no random numbers, and sorting uses the stable keys in the teacher model (Section 8.3). Golden tests depend on this.

15.7 Failure handling

Failure	Behaviour
A rule throws	That rule's findings are shown as "unavailable". Other rules run. The error is logged with the rule ID and no content.
A measurement can't be computed	The findings that depend on it are suppressed, not guessed.
Style fails to load	Style checks are off. Base checks run. The panel explains why (13.5).
The run is interrupted (for example, the tab is closed)	Nothing is saved. The next run starts from the current document.

---

16. Hint policy and familiarity

16.1 What it does

The hint policy changes how findings are displayed, based on how the user has responded to them before. It never changes what is measured, the severity, or the thresholds.

Its only outputs:

- **Collapsed by default:** a rule's optional findings are collapsed under a heading, and can be expanded with one click.
- **Note:** a single line at the top of the rule's group, such as "You've applied this fix before. Expand to review." Shown once per session.

16.2 Rules

1. **Must findings are never collapsed or hidden.** They are always expanded and always counted toward the badge.
2. **Optional findings are collapsed only when** the user has applied at least 5 fixes for that rule, and at least 80% of their responses to that rule were applies (not dismissals).
3. **Dismissals are respected.** A dismissed finding stays hidden until its layers change (teacher model, Section 8.3), regardless of the policy.
4. **No consent, no policy.** Without learning consent, the policy returns defaults, and no counts are used.
5. **Reset.** The user can clear the familiarity counts from Privacy, which resets the policy at once.

16.3 Familiarity counts

Counts are kept per rule, not per layer or per design:

```
rule, applied, dismissed, undone_within_60s, last_updated
```

They are stored in IndexedDB on the device. They are uploaded only if learning consent is granted, under the pseudonymous ID (backend spec, learning endpoints). The server never runs the policy. The policy runs on the device.

16.4 What is recorded

Only outcomes of findings, never content:

Field	Example
`rule`	`contrast`
`class`	`base`
`outcome`	`applied`, `dismissed`, `undone`
`mode`	`corporate_full`
`style_id`	`bauhaus` (or null)
`occurred_at`	timestamp

No layer IDs, no text, no colours, no page content. `undone` means the user undid an applied fix within 60 seconds.

---

17. AI critique integration

17.1 Who triggers it

Only the user, with the "Review my design" action in the Corporate panel. It is never triggered by the rules, the scheduler, or a mode switch.

17.2 Preconditions (checked before the request is built)

Check	Where	If it fails
Level is Corporate	engine	Button not shown
Critique consent granted	browser (from consent state)	Consent prompt is shown first
Feature flag on	browser (from the backend's config response)	Button shows "Coming soon"
Local cache has a review for this image hash	browser	Shows the cached review, no request
Daily limit not reached	browser (from last response)	Shows "2 reviews per day" and when it resets

Server-side checks in the backend spec still apply. These client checks only avoid unnecessary requests.

17.3 Request

The engine produces:

- the page PNG, at most 1024 px on the longest side, with no selection handles or guides
- the page facts: width, height, mode, and the style ID and version, or null
- the findings summary: for each current finding, its rule, class, severity, measured value, and expected value (no layer IDs)
- the locale for the reply language (English only in v1)

The browser sends these to `POST /v1/critique` with the user's consent state. The request body is JSON with the PNG encoded as base64.

17.4 Response handling

Status and code	What the user sees
`200` with a valid review	The review, in the Review section
`feature_disabled`	"Review is paused." (budget or flag)
`consent_required`	Consent prompt
`quota_exceeded`	"2 reviews per day. Try again tomorrow."
`image_too_large`	Should not happen (the engine caps the size). Logged as an error.
`unavailable`, `timeout`	"Review unavailable. Your rules checks still work."
`invalid_output`	Same as unavailable. The server retried once.

The client validates the review's shape again before display, and shows the text as plain text only.

17.5 While waiting

- A loading state with a cancel button. Cancelling stops the client's wait. The server still settles its reservation (backend spec, 9.4).
- The rules panel stays fully usable during the wait.

17.6 Where the review lives

- The review is held in memory for the session only.
- It is not saved in the `.zph` file, not exported, and not written to the event log.
- Closing the design clears it. Reopening and asking again uses the 24-hour cache on the server, which costs nothing.

17.7 Presentation rules

- Points are grouped under "Review", separate from findings.
- A point that refers to a finding shows a "Show" link to that finding. It never has a Fix button.
- The section is labelled as AI text, and can be hidden with one control.
- If a point seems to contradict a measured finding, the contradiction is shown, not removed.

---

18. Workflow and backend touchpoints

This section ties the teacher to the rest of the product, so nothing is left to be decided later.

18.1 Opening and closing a design

Event	Teacher behaviour
Open a design	No run. Mode, style, and last open panel are restored. Dismissals are restored.
Open a Creative design with a style	Style is shown as attached but inactive. No run.
Close a design	Caches and reviews are discarded. Dismissals and style are saved with the design.

18.2 Saving and dismissals

- Dismissed finding keys are stored in the design's metadata, not its content. Each key holds the layer versions at dismissal, so the dismissal ends when those layers change.
- Dismissals sync with the design's metadata on the backend.
- Duplicating a design copies the style, not the dismissals and not the reviews.

18.3 Exporting

- PNG, JPEG, and PSD exports contain only design pixels. Findings, guides, selection handles, and review text are never included.
- An export doesn't count as a check, and doesn't trigger one.

18.4 Trash and restore

Trashing or restoring a design has no effect on its findings or style. Trashed designs cannot be reviewed (backend spec, shares and reviews stop).

18.5 Mode and undo

Change	Undoable	Teacher effect
Switch Creative ↔ Corporate	No	Findings appear or clear
Attach, detach, or change style version	Yes	Checks re-run
Apply a fix	Yes	Checks re-run for affected layers
Apply all	Yes, one step	Checks re-run, up to three passes
Dismiss a finding	No	Finding hidden

18.6 Backend endpoints used by the teacher

Endpoint	Used for	Notes
`GET /v1/catalog/index`	Browse and search styles	Cached, refreshed at most daily
`GET /v1/catalog/styles/:id/:version`	Load a profile	Immutable, cached forever by hash
`GET /v1/catalog/examples/...`	Example images	Static, long cache
`POST /v1/critique`	AI review	Section 17
`GET /v1/consent`, `POST /v1/consent`	Critique and learning consent	Append-only ledger
`POST /v1/learning/events`	Batched outcomes	Only with learning consent, every 60 s or 50 events
`GET /v1/config`	Feature flags, limits, and critique availability	Cached per session

The teacher works without any of these, except the critique and the catalog for styles not yet downloaded.

18.7 Offline behaviour

Feature	Offline
Base checks	Works
Pattern checks for a downloaded style	Works
Pattern checks for a style not downloaded	Off, with message
Style browsing	Works from the cached index, with a note that it may be out of date
AI review	Unavailable
Event upload	Queued, sent on reconnect

18.8 Privacy controls that touch the teacher

- **Critique consent:** withdrawing hides the Review button at once.

- **Learning consent:** withdrawing stops recording and uploading, deletes the local counts, and turns the hint policy to defaults.
- **Export my data:** includes familiarity counts and consent history, not reviews (they are not stored).

---

19. Decisions to confirm

The original decisions (Section 13 of the earlier version) are numbered 1 to 9. The new ones follow.

#	Question	Proposed answer
1	Crop check split	Text and logos are base. Other objects are professional.
2	Margin	Pattern only.
3	Overall fit score	Hidden until the product decides. Per-pattern scores shown.
4	Critique in Corporate, no style	Available, base findings only.
5	Critique in Creative	Not available.
6	"Fires all at once"	One action runs every check and the critique.
7	Professional checks without a style	Off.
8	Print checks	Only when intent is print. Intent set in document settings.
9	Thresholds	Starting values. Confirm after calibration.
10	Style attach and detach	Commands, undoable, stored in document metadata.
11	Style versions	Designs pinned. Updates are an explicit, undoable action.
12	Archived style versions	Kept while referenced. Marked deprecated, never deleted while in use.
13	Style examples	Each needs a licence and attribution, or is rejected.
14	Profile validation	Engine checks the profile on load. Failures are not used.
15	Teacher threading	Main thread in the MVP, time-sliced. Workers later.
16	Hint policy	Display only. Never affects must findings.
17	Familiarity counts	On device. Uploaded only with learning consent.
18	Critique text	Session only. Not saved or exported.
19	Fit preview on style cards	Optional. Build later if the cost is acceptable.
20	Dismissals	Stored in design metadata, tied to layer versions.
21	Exports	Never include findings, guides, or reviews.

20. Build order

1. Measurement registry and the engine's snapshot API, with tests on fixtures.
2. Background sampling and colour measurement, tested on known palettes.
3. Base checks in this order: contrast, text size, crop of text, alignment, text overlap, leading, distortion.
4. Finding model, classification, the store with diffing and dismissals, and the scheduler with the gesture gate.
5. Apply, single fix, and Apply all, with the three-pass limit and the summary step.
6. Style profile format, loader, validator, and compiled cache.
7. Style attach, detach, and version commands, with document metadata and undo.
8. Pattern scoring and the Bauhaus profile, validated against reference designs.
9. Professional checks, shown only with a style.
10. Hint policy and familiarity counts, with learning consent gating.
11. Critique client: request building, response validation, consent and flag checks, session-only display.
12. Catalog sync: index, profile cache by hash, offline states.
13. Calibration: label designs, tune thresholds, record precision and recall.

Gate for step 13: the acceptance figures in Section 10.2 are met for must findings before any must findings are shown to users.